La Sénia
Institut d'Educació Secundària
Paiporta

Unión Europea
Fondo Social Europeo
*El FSE invierte en tu futuro*

# Estructuras de Programación

Curso 2025 – 2026

# 1. Estructuras de Programación

- La Programación Estructurada es un modelo de programación basado en 3 **estructuras de control**.

- Las **estructuras de control** facilitan las tareas de mantenimiento, la reutilización de código y la mejora y corrección de errores.

# 2. Estructuras de control

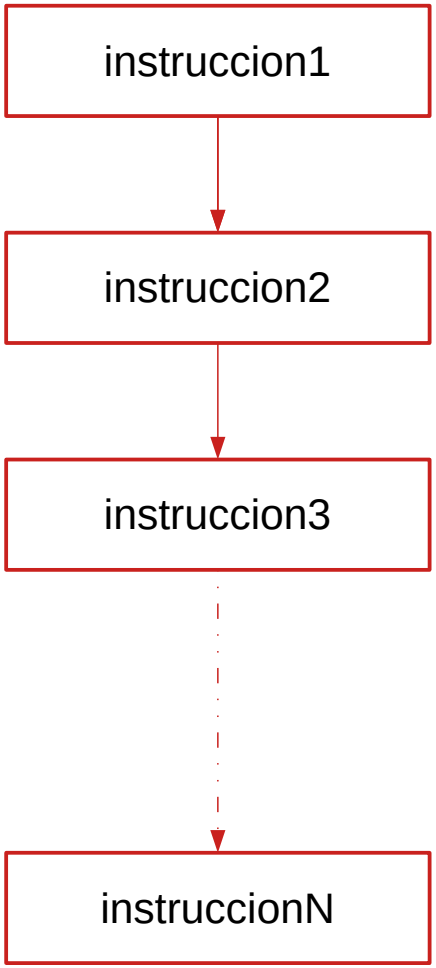
Las **estructuras de control** permiten modificar el flujo de ejecución de un programa.

Permiten adaptar el algoritmo a la realidad.

Todas las estructuras de control tienen **un único punto de entrada**.

- **Secuenciales**: las instrucciones se ejecutan una tras otra
- **Alternativa o selección**: se ejecutan unas acciones u otras dependiendo del cumplimiento de la **condición**
- **Repetitiva o iterativa**: Se ejecutan instrucciones repetidamente. Un número determinado de veces o **mientras** se cumpla una condición.

# 3. Estructura secuencial




```
instruccion1;

instruccion2;

instruccion3;

...

instruccionN;
```

# 3. Estructura secuencial

- Las sentencias del programa se ejecutan de forma **secuencial:** una tras otra y en el mismo orden escrito.
- Cada una de las sentencias o instrucciones están separadas por el carácter punto y coma (**;**).
- Las sentencias se pueden agrupar en bloques. **En Java**, los bloques se encierran entre llaves (**{ }**).
- Si el bloque tiene una única sentencia no es obligatorio, aunque sí recomendable.

Ejemplo:

```
{
    instrucción 1;
    instrucción 2;
    instrucción 3;
}
```

**Ejemplo1:**

**Escribe un programa que pida 2 números enteros y los muestre por pantalla.**

# 3. Estructura secuencial

```java
import java.util.Scanner;

public class Secuencial {

    public static void main(String[] args){
        //Declaración de variables
        int n1, n2;
        Scanner sc = new Scanner(System.in);
        //Leer el primer número
        System.out.println("Introduce un número entero: ");
        n1 = sc.nextInt();
        //Leer el segundo número
        System.out.println("Introduce otro número entero: ");
        n2 = sc.nextInt();
        //Mostrar resultado
        System.out.println("Los números son: " + n1 + " y " + n2);
    }
}
```

# 4. Estructuras de selección

- La **estructura de selección** o condicional permite al programa bifurcar el flujo de ejecución dependiendo de una expresión (**condición**).
- La condición se evalúa como booleano (true o false).
- Se puede diferenciar entre **selección simple**, **selección doble** y **selección múltiple**.

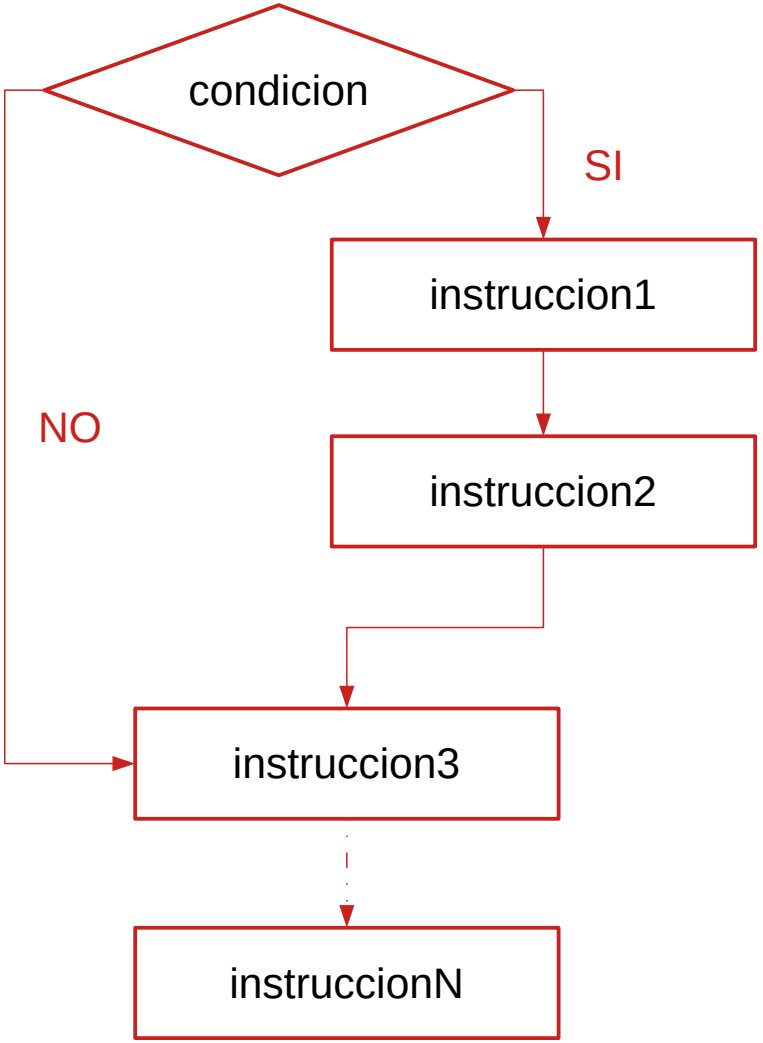
En Java:

Selección simple: if …

Selección doble: if … else … u operador condicional ? :

Selección múltiple: if … else if … else o usando switch case

# 4. Estructuras de selección




```
si condicion entonces
    instruccion1;
    instruccion2;
fsi
instruccion3;
...
instruccionN;
```

# 4. Estructuras de selección

## **Selección simple**:

- Se evalúa la condición y si ésta se cumple se ejecuta una determinadas instrucciones. En caso contrario se saltan esas instrucciones.

```
if (condicion) {
   instrucción 1
   instrucción 2
   .......
}
```

Nota: **Siempre usar las llaves { }** para todos los bloques aunque sólo tenga una instrucción

# Ejemplo2:

**Programa que pide la edad al usuario y determina si es mayor de edad.**

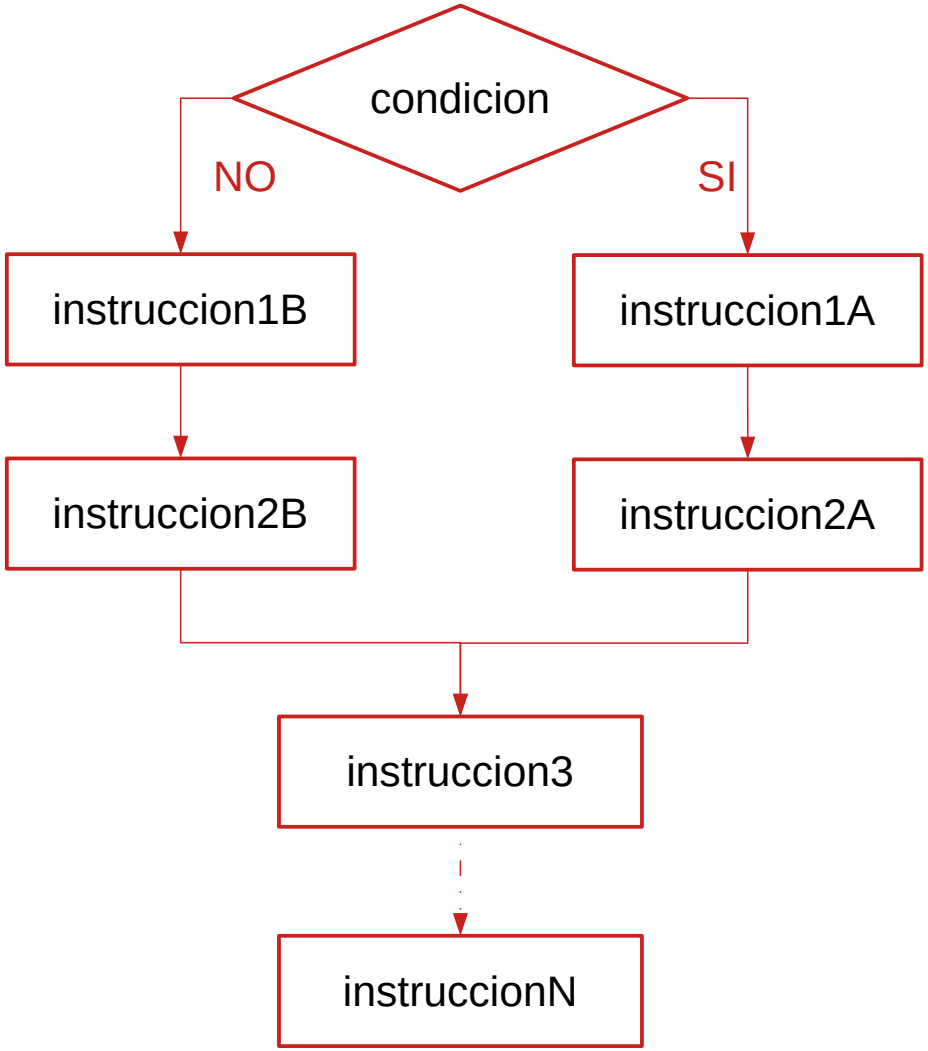
# 4. Estructuras de selección

```java
import java.util.Scanner;

public class MayorEdad {

    public static void main(String[] args){
        //Declaración de variables
        int edad;
        Scanner sc = new Scanner(System.in);
        //Leer la edad
        System.out.println("Introduce tu edad: ");
        edad = sc.nextInt();
        //Comprobar si es mayor de edad
        if (edad >= 18){
            System.out.println("Eres mayor de edad");
        }
    }
}
```

# 4. Estructuras de selección




```
si condicion entonces
        instruccion1A;
        instruccion2A;
sino
        instruccion1B;
        instruccion2B;
fsi
instruccion3;
...
instruccionN;
```

# 4. Estructuras de selección

**Selección doble**:

- Se evalúa la condición y si se cumple se ejecutan unas instrucciones. Si NO se cumple se ejecutan otras instrucciones.

```
if (condicion) {
   instrucciones1
}else{
   instrucciones2
}
```

**Operador condicional** '? :':

Es un operador ternario, igual que if-else, pero en **una instrucción**.

**<condición> ? <InstruccionTrue> : <InstruccionFalse>;**

# Ejemplo3:

**Programa que pide por teclado un número y devuelve si es par o impar.**

# 4. Estructuras de selección

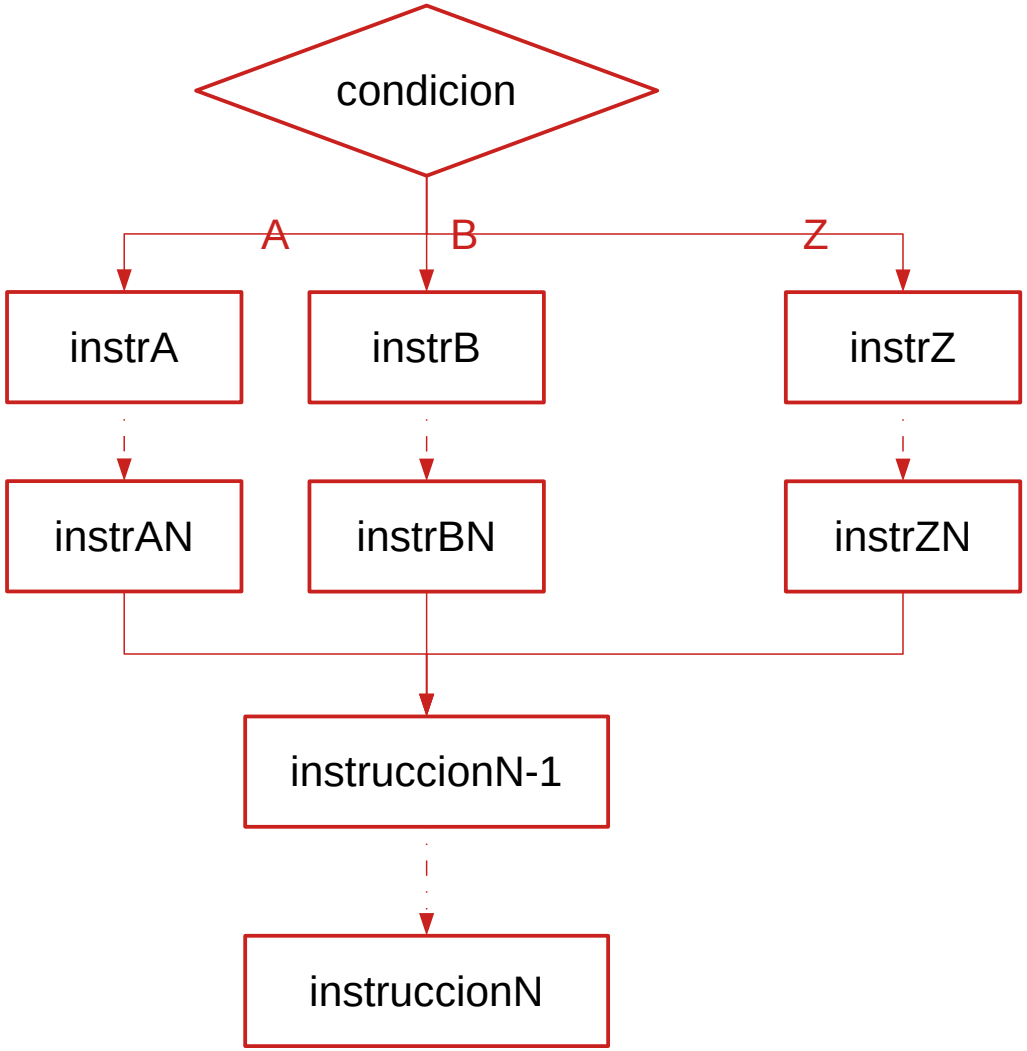
```java
import java.util.Scanner;

public class Par {

    public static void main(String[] args){
        //Declaración de variables
        int num;
        Scanner sc = new Scanner(System.in);
        //Leer el número
        System.out.println("Introduce el número: ");
        num = sc.nextInt();
        if (num % 2 == 0){
            System.out.println("El número " + num + " es par");
        }else{
            System.out.println("El número " + num + " es impar");
        }

        //System.out.println((num%2)==0 ? "Es PAR" : "Es IMPAR");
    }
}
```

# 4. Estructuras de selección




```
caso condicion de
    A:
        InstrA;
        ...
        instrAN;
    B:
        InstrB;
        ...
        instrBN;
    ...
    Z:
        InstrZ;
        ...
        instrZN;
fcaso
instruccionN-1;
...
instruccionN;
```

# 4. Estructuras de selección

## **Selección múltiple**:

- se obtiene anidando sentencias if-else. Permite construir estructuras de selección más complejas.

```
if (condicion) {
   Instrucciones1
}else if(condicion){
   Instrucciones2
}else{
   Instrucciones3
}
```

- Cada **else** se corresponde con el if más próximo que no haya sido emparejado.

# Ejemplo4:

# Programa que muestra un saludo según la hora indicada (mañana/tarde/noche).

# 4. Estructuras de selección

```
import java.util.Scanner;

public class Saludo {

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int hora;

        System.out.println("Introduzca una hora (un valor entero): ");
        hora = sc.nextInt();
        if (hora >= 0 && hora < 12)
            System.out.println("Buenos días");
        else if (hora >= 12 && hora < 21)
            System.out.println("Buenas tardes");
        else if (hora >= 21 && hora < 24)
            System.out.println("Buenas noches");
        else
            System.out.println("Hora no válida");

    }
}
```

## Selección múltiple – Instrucción SWITCH:

- Se utiliza para seleccionar una de entre múltiples alternativas.

```
switch (expresión){
   case valor1:
      instrucciones;
      break;
   case valor2:
      instrucciones;
      break;
   default:
      instrucciones;
}
```

1. *Se evalúa la **expresión** y según su valor salta al **case** correspondiente.*
2. *Se ejecutan las **instrucciones** que siguen al case seleccionado hasta que se encuentra un **break** o el final del switch.*
3. *Si ningún **case** se cumple se ejecuta el bloque **default** (si existe). No es obligatorio y no tiene porqué ponerse siempre al final, aunque es lo habitual.*
4. *Clausula **break** para salir de la instrucción múltiple*

- Se puede usar con datos de tipo byte, short, char e int.
- También con tipos enumerados y clases (Character, Byte, Short, Integer y String)

# Ejemplo5:

# Programa que pide el número del mes y muestra el nombre correspondiente a ese mes

# 4. Estructuras de selección

```
import java.util.Scanner;

public class Meses {
    public static void main(String[] args) {
        int mes;
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduzca un numero de mes: ");
        mes = sc.nextInt();
        switch (mes){
            case 1: System.out.println("ENERO"); break;
            case 2: System.out.println("FEBRERO"); break;
            case 3: System.out.println("MARZO"); break;
            case 4: System.out.println("ABRIL"); break;
            case 5: System.out.println("MAYO"); break;
            case 6: System.out.println("JUNIO"); break;
            case 7: System.out.println("JULIO"); break;
            case 8: System.out.println("AGOSTO"); break;
            case 9: System.out.println("SEPTIEMBRE"); break;
            case 10: System.out.println("OCTUBRE"); break;
            case 11: System.out.println("NOVIEMBRE"); break;
            case 12: System.out.println("DICIEMBRE"); break;
            default : System.out.println("Mes no válido");
        }
    }
}
```

# Ejercicios:

- 663. Contando desde el cero
- 715. ¿Hay suficientes?
- 356. Dólar Zimbabuense
- 626. Plinio el joven
- 657. De las pirámides a Cleopatra
- 751. Coetaneos
- 112. Radares de tramo

# 5. Estructuras de repetición

- Permiten ejecutar de forma repetida un bloque específico de instrucciones.
- Las instrucciones se repiten mientras se cumpla una condición, llamada **condición de salida**.
- La condición de salida debe cambiar para evitar una **repetición infinita.**
- Tipos de estructuras repetitivas:

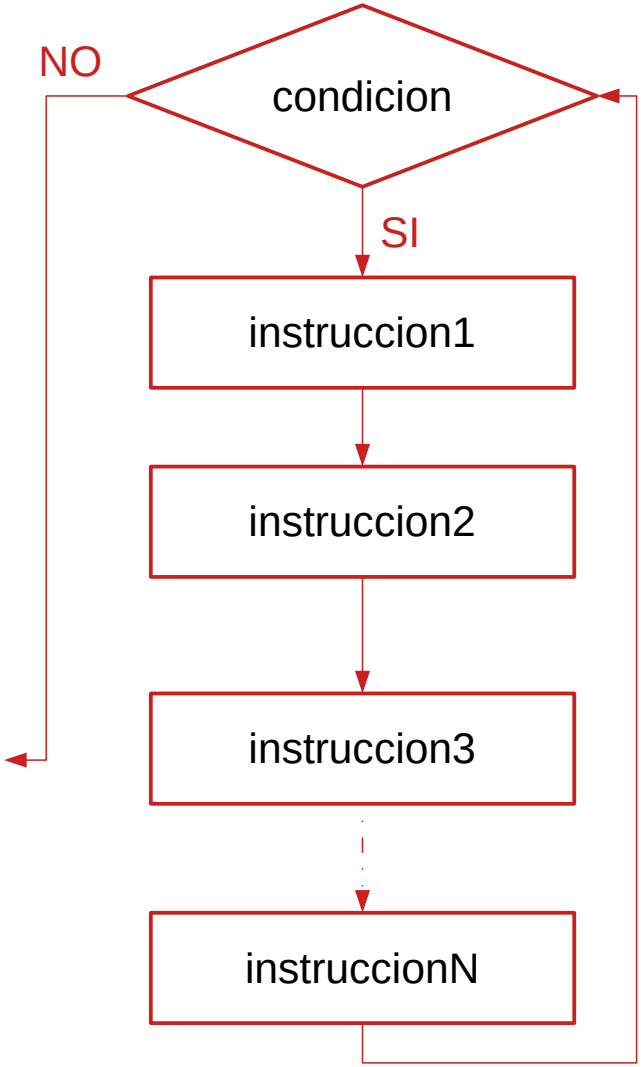
  **while**

  **do - while**

  **for**

  **foreach**

# 5. Estructuras de repetición (while)




**mientras** condicion **hacer**

instruccion1;

instruccion2;

instruccion3;

...

instruccionN;

**fmientras**

# 5. Estructuras de repetición (while)

**while**:

- Las instrucciones se repiten mientras la condición sea verdadera.
- **La condición se comprueba al principio** por lo que las instrucciones podrían no ejecutarse nunca (si la condición es inicialmente falsa) ó más veces.

```
while (condicion){
    instrucciones;
}
instrucciones;
```

1. *Se evalúa la **condición**.*
2. *Si el resultado de la condición es **true** se ejecutan las instrucciones de dentro del while y se vuelve al paso 1.*
3. *Si condición es **false** el bloque no se ejecuta y el programa continúa con las instrucciones a continuación del while.*

**Ejemplo6:**

Programa que imprime las líneas de un fichero. Lo recorre hasta que no quede ninguna por imprimir.

# 5. Estructuras de repetición (while)

```
import java.util.Scanner;
import java.io.*;

public class ImprimirFichero{

    public static void main(String[] args) throws FileNotFoundException{

        Scanner sc = new Scanner(new File("fichero.txt"));
        String s = "";

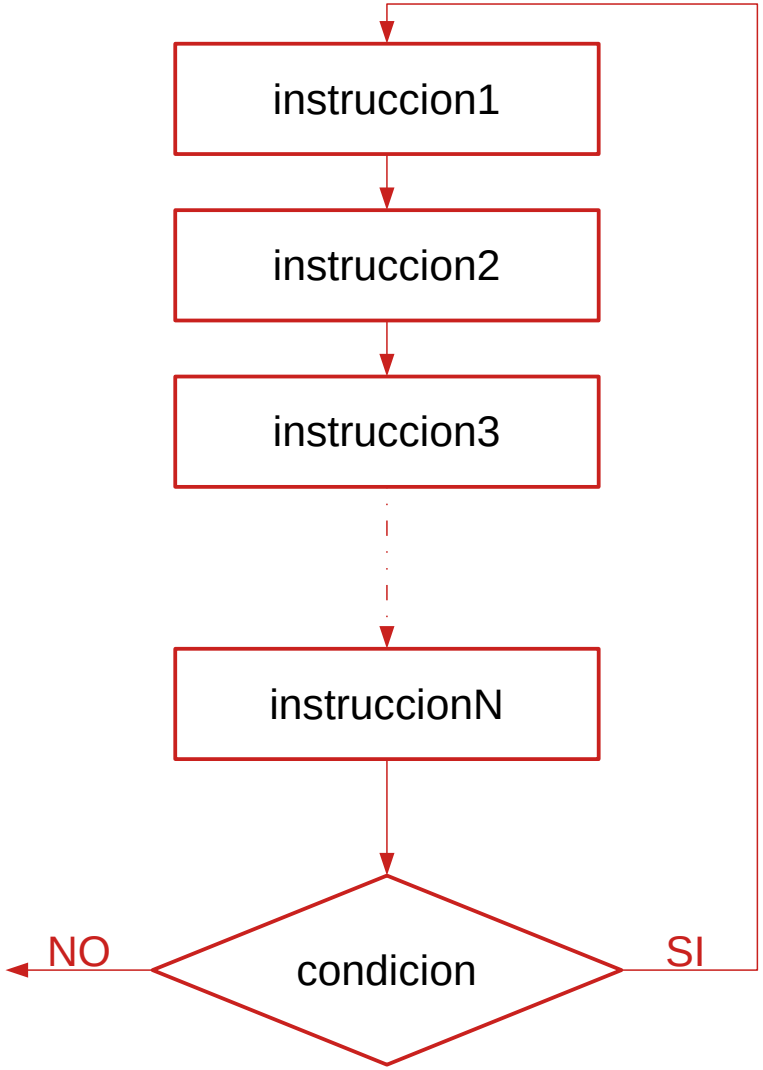
        //Mientras hay líneas por leer…
        while(sc.hasNext()){

            //…leer la línea del fichero…
            s = sc.nextLine();

            //…e imprimirla por pantalla.
            System.out.println( s );

        }
    }
}
```

# 5. Estructuras de repetición (do..while)




**repetir**

    instruccion1;

    instruccion2;

    instruccion3;

    ...

    instruccionN;

**mientras** condicion

# 5. Estructuras de repetición (do..while)

**do ... while**:

- Las instrucciones se ejecutan mientras la condición sea cierta.
- La condición se comprueba al final del bucle por lo que el bloque de instrucciones se ejecutarán al menos una vez, aunque la condición sea falsa.

```
do {
    instrucciones;
} while (condicion);
instrucciones;
```

1. *Se ejecutan todas las instrucciones a partir de **do{** hasta llegar a la condición.*
2. *Se evalúa la **condición** de **while**.*
3. *Si el resultado es **true** se vuelve al paso 1.*
4. *Si el resultado es **false**, el programa sigue la ejecución con las siguientes instrucciones.*

# 5. Estructuras de repetición (do..while)

**Ejemplo7:**

Programa que lee e imprime números hasta que se introduce un 0.

# 5. Estructuras de repetición (do..while)

```
import java.util.Scanner;

public class Hasta0 {

    public static void main(String[] args) {
        int num;
        Scanner sc = new Scanner(System.in);

        do {

            System.out.print("Introduzca un numero: ");

            num = sc.nextInt();

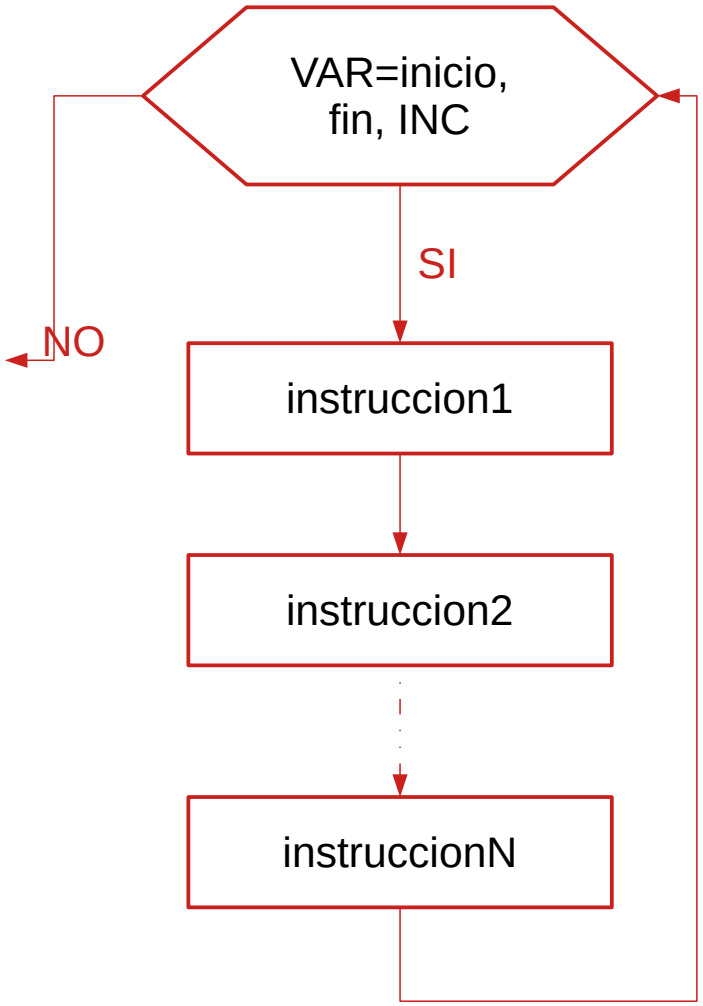
            System.out.print("Has introducido un " + num);

        } while (num != 0);

        System.out.print("FIN DEL PROGRAMA");

    }
}
```

# 5. Estructuras de repetición (for)




**para** VAR **desde** inicio **hasta** fin
[ **con incremento** INC ] **hacer**

instruccion1;

instruccion2;

instruccion3;

...

instruccionN;

**fpara**

# 5. Estructuras de repetición (for)

**for**:

- Hace que una instrucción o bloque de instrucciones se repitan un número determinado de veces, mientras se cumpla la condición.

```
for( inicialización; condición;incremento/decremento){
    instrucciones;
}
instrucciones;
```

1. *Se inicializa la **variable de control** (inicialización)*
2. *Se evalúa la **condición**.*
3. *Si el resultado es **true**, se ejecutan las instrucciones del bucle.*
4. *Al final del bucle se actualiza la variable de control (**incremento/decremento**) y se vuelve al punto 2.*
5. *Si el resultado es **false**, el programa continúa con las instrucciones después del for.*

# 5. Estructuras de repetición (for)

- A continuación de la palabra **for** y entre paréntesis debe haber siempre **tres zonas separadas**:
  - **Inicialización:** La/s variable/s de control del bucle toman su valor inicial. Se realiza una vez y puede haber una o más variables inicializadas.
  - **Condición:** Mientras sea cierta ejecuta el contenido del bucle. Generalmente, la condición compara las variables de control con un valor límite.
  - **Incremento/decremento:** Decrementa o incrementa la/s variable/s de control del bucle.

```
for(int x = 0, y = 1; x < 100 && y < 100; x=x+1, y=y+2) {
    . . .
 }
```

**Ejemplo8:**

Función que imprime los 50 primeros números enteros.

# 5. Estructuras de repetición (for)

```
public class Imprime50 {

    public static void main(String[] args) {

        //Recorre imprimiendo los 50 numeros

        for (int i = 1; i <= 50; i++) {

            //Imprime el número i
            System.out.println( i );

        }
    }
}
```

# 5. Estructuras de repetición (foreach)

## foreach (for extendido):

- Esta forma de usar el for es muy util para recorrer los objetos de una colección sin especificar el número de elementos.

```
for (TipoARecorrer nombreVariable : nombreDeLaColección ) {
   instrucciones
}
```

```
public class ForEach { // Nuestra clase principal o programa
    public static void main(String[] args) { // Función principal a ejecutar
        String[] listaNombres = { "Alex", "Ana", "Luisa", "Jose" };
        for (String nombre : listaNombres) {
            System.out.println(nombre);
        }
    }
}
```

# 5. Estructuras de repetición

- Java permite la posibilidad de construir bucles infinitos, los cuales se ejecutarán indefinidamente, a no ser que provoquemos su interrupción. Tres ejemplos:

```
for(;;){
    instrucciones
}

for(;true;){
    instrucciones
}

while(true){
    instrucciones
}
```

**¡¡¡¡Cuidado!!!!**

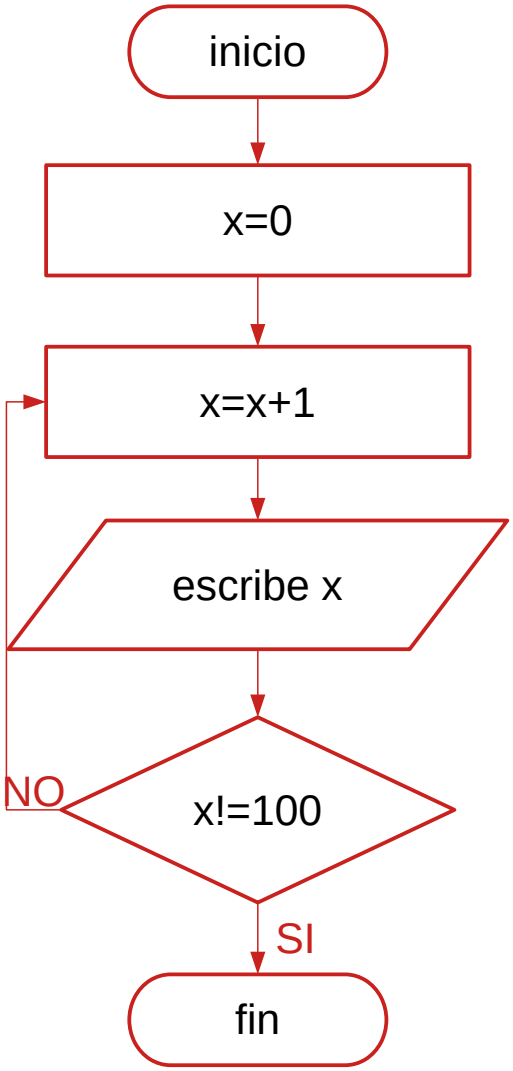
*Los bucles infinitos también suelen producirse de forma involuntaria al no modificar correctamente la condición de salida de un bucle.*
*Un ERROR muy común para el programador novel.*

**Bucles anidados**:

- Son aquellos que incluyen instrucciones for, while o do-while unas dentro de otras.
- Debemos tener en cuenta que las variables de control que utilicemos deben ser distintas.
- Los anidamientos deben mantener una estructura anidada dentro de otra completamente.

```
for(int i = 0; i < 10; i++) {
    for(int j = 0; j < 10; j++) {
        System.out.println("El número es "+i+j);
    }
    System.out.println("Fin del programa");
}
```

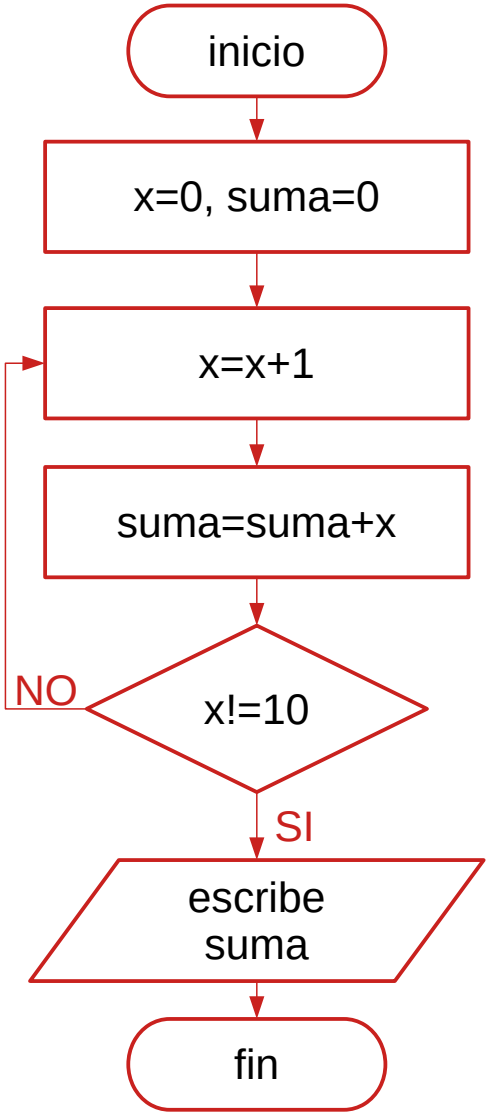
# 6. Variables especiales (contador)




```
inicio
    x=0;
    repetir
        x=x+1;
        escribe x;
    mientras (x!=100)
fin
```

**x** es una variable ***contador*** que indica el número de veces que hemos ejecutado el bucle

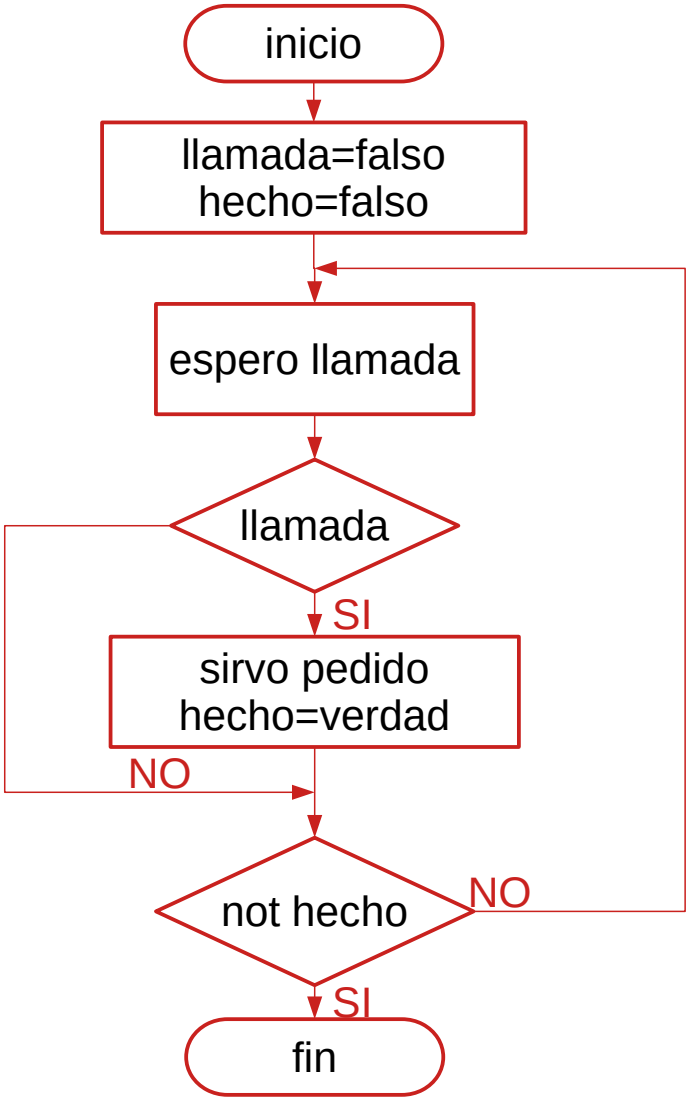
# 6. Variables especiales (acumulador)




```
inicio
    x=0;
    suma=0;
    repetir
        x=x+1;
        suma=suma+x;
    mientras (x!=10)
    escribe x;
fin
```

**suma** es una variable ***acumulador*** que almacena la suma de los valores del contador x cada vez que ejecutamos el bucle

# 6. Variables especiales (interruptor)




```
inicio
    llamada=false;
    hecho=false;
    repetir
        si llamada entonces
            sirvo pedido;
            hecho = true;
        fsi;
    mientras not hecho (condición de fin)
fin
```

La variable ***interruptor*** **hecho** indica la condición de fin e interrumpe la ejecución del bucle

# 7. Estructuras de Salto

- Las **estructuras de salto** son instrucciones que permiten romper el orden natural de ejecución de nuestros programas, pudiendo saltar a un determinado punto o instrucción.
- Las instrucciones de salto son: **break**, **continue** y **return**
- En estructuras de repetición:
  - para salir de forma abrupta de un bucle (**break**).
  - para finalizar la iteración y evaluar la siguiente (**continue**).
- **break** se usa para interrumpir la ejecución de un switch.
- **return** se utiliza para finalizar una función o método.

# 7. Estructuras de Salto

**break**:

- Se utiliza para **interrumpir la ejecución** de una estructura de repetición o de un switch.
- El flujo continúa en la sentencia inmediatamente posterior a la estructura de repetición o switch.

```
public class SaltoBreak {
    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            System.out.println("Dentro del bucle "+i);
            break;
            System.out.println("Nunca lo escribirá");
        }
        System.out.println("Tras el bucle");
    }
}
```

# 7. Estructuras de Salto

**continue**:

- Aparece dentro de una estructura de repetición.
- Finaliza la iteración para volver a evaluar la condición inicial y continuar con la siguiente iteración.

```
public class SaltoContinue {
    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            System.out.println("Dentro del bucle "+i);
            continue;
            System.out.println("Nunca lo escribira");
        }
        System.out.println("Tras el bucle");
    }
}
```

# 7. Estructuras de Salto

Uso inadecuado del salto.

- Usar el salto para finalizar la ejecución de un bucle infinito.

```
public class UsoBreakNo {
    public static void main(String[] args) {
        while (true) {
          System.out.println("Introduce numero: ");
          num = sc.nextInt();
          if (num == 42){
            System.out.println("ACIERTO");
            break;
        }
        System.out.println("FALLO");
    }
}
```

# Ejercicios:

- 149. San Fermines
- 158. Los saltos de Mario
- 191. Los problemas de ser rico
- 265. Suma descendente
- 586. Colección de calendarios de bolsillo
- 743. Mejor a doble cara
- 773. Reto de polvorones